

16. Implementation Details

16.1 Repository Organisation

The project is a single repository containing five deployable or executable concerns, separated by directory rather than by package boundary.

```
behavioral-risk-assessment/  
  data/  
    raw/          generate_dataset.py, dataset.csv, dataset_description.md  
    processed/    preprocess.py, processed_dataset.csv, scaler.pkl, plots/  
  ml/  
    train.py, inference.py, test_inference.py  
    models/       risk_model.pkl, investment_model.pkl, label encoders,  
                  feature_columns.json, model_metadata.json,  
                  preprocessing_params.json  
    plots/        shap_summary.png, feature_importance.png  
  backend/  
    app/          main.py, config.py, database.py,  
                  routers/, services/, schemas/, models/, middleware/  
    Dockerfile, requirements.txt, .env.example  
  frontend/  
    src/          api/, pages/, components/, contexts/, types/  
    Dockerfile, nginx.conf, package.json  
  docs/  
    api-contract.json, report/  
  tests/  
    test_api.py, test_inference.py  
  docker-compose.yml
```

The `data` and `ml` directories are executed offline and produce artefacts. The `backend` and `frontend` directories are long-running services. Nothing in `backend` writes to `ml`; the dependency runs one way only.

16.2 Toolchain

Layer	Technology	Version
Frontend framework	React	19
Language	TypeScript	6.0
Build tool	Vite	8
Styling	TailwindCSS	3.4
Charts	Recharts	3.9
Forms and validation	React Hook Form 7, Zod 4	
Routing	React Router	7
Backend framework	FastAPI	≥ 0.110
ORM	SQLAlchemy	2.0
Validation	Pydantic	2
Database	PostgreSQL 15 / SQLite 3	
ML	scikit-learn, XGBoost, LightGBM, CatBoost, SHAP	
Orchestration	Docker Compose	

Radix UI primitives underpin the select, dialog, progress and tooltip components. Linting uses `oxlint`.

16.3 Configuration

Runtime configuration is read from environment variables through a `pydantic-settings` model, with `.env` as a fallback source and defaults suitable for local development.

Variable	Default	Purpose
<code>DATABASE_URL</code>	<code>sqlite:///./risk_assessment.db</code>	connection string
<code>ML_PATH</code>	<code>/ml</code>	location of the inference module
<code>SECRET_KEY</code>	<code>dev-secret-key</code>	declared, currently unused
<code>LOG_LEVEL</code>	<code>INFO</code>	logging verbosity
<code>VERSION</code>	<code>1.0.0</code>	reported by <code>/health</code>

`SECRET_KEY` is defined in the settings model but referenced nowhere in the codebase. It was provisioned for a session or token mechanism that was not implemented, and it is retained only because removing it would require a corresponding change to the deployment templates. It signs nothing.

16.4 Deployment

Three containers are composed. PostgreSQL exposes 5432 and declares a `pg_isready` health check; the backend declares `depends_on` with `condition: service_healthy`, so it does not start against a database still initialising. The backend mounts `./ml` at `/ml` — this bind mount is what makes the model available to the service, and the container will start without it but report `model_loaded: false` from `/health` and return 503 from `/predict`.

The frontend is built in a multi-stage image: a Node stage compiles the TypeScript and produces static assets; an nginx stage serves them. The nginx configuration proxies `/api/` to the backend service and rewrites unmatched paths to `index.html`, which is required for client-side routing to survive a page reload.

Two adjustments were necessary to make the compose stack build. The frontend Dockerfile originally invoked `npm ci`, which aborts when `package-lock.json` and `package.json` disagree; the lockfile in the repository had drifted, and the instruction was changed to `npm install`. Separately, the backend image did not install NumPy, pandas, scikit-learn or SHAP, because `requirements.txt` listed only the web dependencies. The ML packages were added, since the backend imports the inference module directly into its own process.

16.5 Offline Pipeline Execution

The three offline scripts are run in sequence and are individually idempotent under their fixed seed.

```
cd data/raw      && python3 generate_dataset.py    # → dataset.csv
cd ../processed  && python3 preprocess.py          # → processed_dataset.csv
                                                         # preprocessing_params.json
cd ../../ml      && python3 train.py                # → models/*.pkl, metadata
```

Re-running `preprocess.py` after the parameter-persistence change was verified to leave `processed_dataset.csv` byte-identical, which is the reason the existing trained model remained valid and no retraining was required when the serving defects described in Section 22 were repaired.

16.6 Testing

Eleven tests exist across two files.

`tests/test_inference.py` holds five unit tests exercising `predict()` directly, without a server. They assert that all twelve expected keys are present in the returned dictionary, that `risk_category` is one of the three permitted labels, that both confidence values lie in `[0, 1]`, that the three scores lie in `[0, 100]`, and that at least one recommendation is produced.

`tests/test_api.py` holds six integration tests requiring a running service. They cover `/health`, `/model/info`, a successful `/predict`, an assessment round trip through `POST /assessment` followed by `GET /assessment/{id}`, the `/analytics` aggregate, and one negative case asserting that an `age` of 10 is rejected with HTTP 422.

Coverage is thin and should be described as such. There are no tests for the preprocessing scripts, none for the frontend, and none that would have caught any of the five feature-engineering defects discussed in Section 22 — because every one of those defects lay in the agreement between two code paths, and no test compared them. The equivalence check that eventually exposed them, described in Section 9.7, is not part of the automated suite. Promoting it to a regression test would be the single most valuable addition to this repository.

16.7 Defects Identified During Documentation

Writing this report required reading the implementation closely, and doing so surfaced defects that the test suite had not. Three are recorded here because they affect the backend as deployed; the five feature-engineering defects are treated in Section 22, and the data-loading defect in Section 12.5.

The `/model/info` endpoint reads an absolute path from the author's machine. `backend/app/routers/health.py` defines `MODEL_METADATA_PATH` as `/Users/chiragmali/Documents/.../ml/models/model_metadata.json`. Inside the container this path does not exist, so the endpoint returns `{"error": "model_metadata.json not found"}` with HTTP 200 rather than the metadata. It works when the service runs directly on the development machine and fails under Docker Compose, which is the documented deployment path. It should resolve the path relative to `ML_PATH`, as `ml_service` does.

The local-development fallback in `ml_service` resolves one directory too high. When `/ml` is absent the module constructs a fallback path by ascending four levels from `app/services/`, which lands on the repository's parent directory rather than the repository root. The resulting path does not exist, the import fails, and `/predict` returns 503. The defect is invisible under Docker Compose, where `/ml` is always mounted and the fallback never executes.

The audit middleware performs a synchronous database write inside an asynchronous handler. `LoggingMiddleware.dispatch` is a coroutine, but it opens a SQLAlchemy session and commits without yielding, which blocks the event loop for the duration of the write on every request. The inline comment describes the write as "non-blocking best-effort"; it is best-effort, in that a failure is swallowed, but it is not non-blocking. The middleware additionally persists the entire request body to `audit_logs`, which for `POST /assessment` means storing twenty-six fields of a user's financial position in a table that carries no access control.

None of the three affects the accuracy figures reported in Section 23, because none touches the prediction path in the configuration under which those figures were produced.

Figure 16.1 — *Repository tree*, rendered as a directory listing. Trim to two levels. Place in Section 16.1.

Screenshot 16.1 — *docker compose up reaching a healthy state*, showing all three containers running and the backend log line `ML models loaded and ready`. Place in Section 16.4.

Screenshot 16.2 — *Terminal output of `python3 -m pytest tests/ -v`*, showing all eleven tests passing. Place in Section 16.6.

Table 16.1 — *Environment variables*, reproducing the table in Section 16.3 with an added column stating whether each is consumed by code. Mark `SECRET_KEY` as unused.